

```

----- Server.js -----
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

app.use(express.static('public'));

let gafetes = {
  'gafete-1': {
    id: 'gafete-1',
    nombreGafete: 'Gafete Principal',
    linea1: 'JUAN PÉREZ',
    size1: 32,
    color1: '#ffffff',
    linea2: 'Presentador / Host',
    size2: 18,
    color2: '#a1a1aa',
    posX: 5,
    posY: 80,
    usarLogo: false,
    logoUrl: '',
    logoSize: 50,
    logoPos: 'left',
    animEntrada: 'slideLeft',
    animSalida: 'slideLeft',
    timerSeconds: 5,
    usarCiclo: false,
    tiempoOculto: 3,
    visibilidad: false
  }
};

// Timers para los ciclos activos en el servidor
const cicloTimers = {};

function gestionarCiclo(id) {
  const g = gafetes[id];
  if (cicloTimers[id]) clearTimeout(cicloTimers[id]);

  if (!g.visibilidad && !g.usarCiclo) return;

  if (g.visibilidad) {
    // Si está visible, programar el ocultamiento tras T1 (timerSeconds)
    const t1 = (g.timerSeconds || 5) * 1000;
    cicloTimers[id] = setTimeout(() => {
      g.visibilidad = false;
    }, t1);
  }
}

```

```

    io.emit('sync-gafetes', gafetes);

    // Si tiene ciclo activo, programar la reaparición tras T2 (tiempoOculto)
    if (g.usarCiclo) {
        gestionarCiclo(id);
    }
}, t1);
} else if (g.usarCiclo) {
    // Si está oculto pero el ciclo sigue activo, esperar T2 para volver a mostrar
    const t2 = (g.tiempoOculto || 3) * 1000;
    cicloTimers[id] = setTimeout(() => {
        g.visibilidad = true;
        io.emit('sync-gafetes', gafetes);
        gestionarCiclo(id);
    }, t2);
}
}

io.on('connection', (socket) => {
    socket.emit('sync-gafetes', gafetes);

    socket.on('guardar-gafete', (data) => {
        gafetes[data.id] = { ...gafetes[data.id], ...data };
        io.emit('sync-gafetes', gafetes);
    });

    socket.on('crear-gafete', () => {
        const id = 'gafete-' + Date.now();
        gafetes[id] = {
            id: id,
            nombreGafete: 'Nuevo Gafete',
            linea1: 'NUEVO TÍTULO',
            size1: 28,
            color1: '#ffffff',
            linea2: 'Subtítulo',
            size2: 16,
            color2: '#cccccc',
            posX: 10,
            posY: 10,
            usarLogo: false,
            logoUrl: '',
            logoSize: 40,
            logoPos: 'left',
            animEntrada: 'fadeIn',
            animSalida: 'fadeOut',
            timerSeconds: 5,
            usarCiclo: false,
            tiempoOculto: 3,
            visibilidad: false
        };
    });

```

```

    io.emit('sync-gafetes', gafetes);
  });

  socket.on('eliminar-gafete', (id) => {
    if (cicloTimers[id]) clearTimeout(cicloTimers[id]);
    delete gafetes[id];
    io.emit('sync-gafetes', gafetes);
  });

  // Toggle de la botonera
  socket.on('toggle-visibilidad', (id) => {
    if (gafetes[id]) {
      gafetes[id].visibilidad = !gafetes[id].visibilidad;
      io.emit('sync-gafetes', gafetes);
      gestionarCiclo(id);
    }
  });

  socket.on('disparar-sonido', (sonido) => {
    io.emit('reproducir-sonido', sonido);
  });
});

server.listen(3000, () => {
  console.log('Servidor listo en http://localhost:3000');
});

----- control.html
-----
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Control de Gafetes Pro - OBS</title>
  <script src="/socket.io/socket.io.js"></script>
  <style>
    body { font-family: system-ui, sans-serif; background: #09090b; color: #f4f4f5;
margin: 0; padding: 15px; }
    .layout { display: grid; grid-template-columns: 320px 1fr 280px; gap: 15px;
max-width: 1400px; margin: auto; }
    .card { background: #18181b; padding: 15px; border-radius: 8px; border: 1px
solid #27272a; }
    h3 { margin-top: 0; font-size: 16px; border-bottom: 1px solid #27272a;
padding-bottom: 8px; color: #3b82f6; }
    label { display: block; margin-top: 8px; font-size: 12px; color: #a1a1aa; }
    input[type="text"], input[type="number"], select { width: 100%; padding: 6px;
margin-top: 2px; background: #09090b; border: 1px solid #27272a; color: #fff;
border-radius: 4px; box-sizing: border-box; font-size: 13px; }
    .row { display: flex; gap: 8px; align-items: center; }
    .checkbox-row { display: flex; align-items: center; gap: 8px; margin-top: 10px;

```

```

cursor: pointer; }
.checkbox-row input { width: auto; cursor: pointer; }
button { cursor: pointer; padding: 8px 12px; border: none; border-radius: 4px;
font-weight: bold; width: 100%; margin-top: 10px; font-size: 13px; }
.btn-blue { background: #2563eb; color: white; }
.btn-green { background: #16a34a; color: white; }
.btn-red { background: #dc2626; color: white; }
.btn-gray { background: #27272a; color: #a1a1aa; border: 1px solid #3f3f46; }
.btn-gray.active { background: #16a34a; color: white; border-color: #22c55e; }

/* PANTALLA DE PREVIO INTERACTIVA */
#stage {
width: 100%;
aspect-ratio: 16/9;
background: #000;
background-image: radial-gradient(#27272a 1px, transparent 1px);
background-size: 12px 12px;
position: relative;
border-radius: 6px;
overflow: hidden;
border: 2px solid #3f3f46;
}
.prev-box {
position: absolute;
padding: 6px 12px;
background: rgba(0, 0, 0, 0.85);
border-left: 4px solid #2563eb;
border-radius: 4px;
cursor: move;
user-select: none;
display: flex;
gap: 8px;
align-items: center;
transform: translate(-50%, -50%); /* Centrado sobre punto X,Y */
max-width: 80%;
}
.prev-box.selected { border-color: #f59e0b; box-shadow: 0 0 10px rgba(245, 158,
11, 0.5); }
.prev-box.logo-top { flex-direction: column; text-align: center; }
.prev-box.logo-bottom { flex-direction: column-reverse; text-align: center; }
.prev-box.logo-right { flex-direction: row-reverse; }

.bot-item { margin-bottom: 8px; padding: 8px; background: #09090b;
border-radius: 4px; border: 1px solid #27272a; }
</style>
</head>
<body>

<div class="layout">
  <!-- COLUMNA 1: CONFIGURACIÓN -->

```

```

<div class="card">
  <h3>⚙ Editar Gafete</h3>
  <label>Gafete Activo:</label>
  <select id="selGafete" onchange="seleccionarGafete(this.value)"></select>
  <button class="btn-gray" onclick="socket.emit('crear-gafete')">+ Crear
Nuevo Gafete</button>

  <label>Nombre Identificador:</label>
  <input type="text" id="nombreGafete" oninput="actualizar()">

  <label>Línea 1 (Título):</label>
  <input type="text" id="linea1" oninput="actualizar()">
  <div class="row">
    <input type="number" id="size1" title="Tamaño px" oninput="actualizar()">
    <input type="color" id="color1" oninput="actualizar()">
  </div>

  <label>Línea 2 (Subtítulo):</label>
  <input type="text" id="linea2" oninput="actualizar()">
  <div class="row">
    <input type="number" id="size2" title="Tamaño px" oninput="actualizar()">
    <input type="color" id="color2" oninput="actualizar()">
  </div>

  <!-- CHECK Y CONFIG DE LOGO -->
  <label class="checkbox-row">
    <input type="checkbox" id="usarLogo" onchange="actualizar()">
    <span><strong>¿Usar Imagen / Logo?</strong></span>
  </label>
  <div id="sec-logo" style="display:none; padding-left: 10px; border-left: 2px
solid #2563eb;">
    <label>URL de la imagen:</label>
    <input type="text" id="logoUrl" placeholder="http://... o
imagenes/logo.png" oninput="actualizar()">
    <div class="row">
      <div><label>Tamaño (px):</label><input type="number" id="logoSize"
value="40" oninput="actualizar()"></div>
      <div>
        <label>Posición Logo:</label>
        <select id="logoPos" onchange="actualizar()">
          <option value="left">Izquierda</option>
          <option value="right">Derecha</option>
          <option value="top">Arriba</option>
          <option value="bottom">Abajo</option>
        </select>
      </div>
    </div>
  </div>

  <!-- ANIMACIONES -->

```

```

<label>Animación Entrada / Salida:</label>
<div class="row">
  <select id="animEntrada" onchange="actualizar()">
    <option value="slideLeft">Deslizar Izq</option>
    <option value="slideRight">Deslizar Der</option>
    <option value="slideUp">Deslizar Arriba</option>
    <option value="bounce">Rebote</option>
    <option value="zoom">Zoom</option>
    <option value="fadeIn">Aparecer (Fade)</option>
  </select>
  <select id="animSalida" onchange="actualizar()">
    <option value="slideLeft">Deslizar Izq</option>
    <option value="slideRight">Deslizar Der</option>
    <option value="slideUp">Deslizar Arriba</option>
    <option value="zoom">Zoom</option>
    <option value="fadeOut">Desaparecer (Fade)</option>
  </select>
</div>

<!-- CICLO DE TRABAJO -->
<label class="checkbox-row">
  <input type="checkbox" id="usarCiclo" onchange="actualizar()">
  <span><strong>Habilitar Bucle / Ciclo de 2 Tiempos</strong></span>
</label>
<div class="row">
  <div><label>T1: Visible (s):</label><input type="number" id="timerSeconds"
value="5" oninput="actualizar()"></div>
  <div><label>T2: Oculto (s):</label><input type="number" id="tiempoOculto"
value="3" oninput="actualizar()"></div>
</div>

  <button class="btn-red" style="margin-top:15px;"
onclick="eliminarActual()">🗑 Eliminar Gafete</button>
</div>

<!-- COLUMNA 2: PREVIO CON DRAG & DROP -->
<div class="card">
  <h3>📺 Previo Interactivo (Arrastra para Ubicar)</h3>
  <div id="stage"></div>
  <div class="row" style="margin-top:8px;">
    <label>Pos $X$ (%): <span id="lblX">0</span></label>
    <label>Pos $Y$ (%): <span id="lblY">0</span></label>
  </div>
</div>

<!-- COLUMNA 3: BOTONERA DE CONTROL -->
<div class="card">
  <h3>🔊 Botonera de Transmisión</h3>
  <p style="font-size:11px; color:#a1a1aa; margin-top:-5px;">Presiona para
Activar/Desactivar el gafete en OBS.</p>

```

```

    <div id="botonera"></div>
  </div>
</div>

<script>
  const socket = io();
  let gafetesLocales = {};
  let idSeleccionado = null;
  let dragging = false;

  socket.on('sync-gafetes', (data) => {
    gafetesLocales = data;
    const keys = Object.keys(data);
    if (keys.length > 0 && !idSeleccionado) {
      idSeleccionado = keys[0];
    }
    render();
  });

  function render() {
    // 1. Selector
    const sel = document.getElementById('selGafete');
    sel.innerHTML = '';
    Object.values(gafetesLocales).forEach(g => {
      const opt = document.createElement('option');
      opt.value = g.id;
      opt.textContent = g.nombreGafete || g.id;
      if (g.id === idSeleccionado) opt.selected = true;
      sel.appendChild(opt);
    });

    // 2. Cargar Formulario del activo
    const g = gafetesLocales[idSeleccionado];
    if (g) {
      document.getElementById('nombreGafete').value = g.nombreGafete || '';
      document.getElementById('linea1').value = g.linea1 || '';
      document.getElementById('size1').value = g.size1 || 28;
      document.getElementById('color1').value = g.color1 || '#ffffff';
      document.getElementById('linea2').value = g.linea2 || '';
      document.getElementById('size2').value = g.size2 || 16;
      document.getElementById('color2').value = g.color2 || '#cccccc';

      const checkLogo = document.getElementById('usarLogo');
      checkLogo.checked = !!g.usarLogo;
      document.getElementById('sec-logo').style.display = g.usarLogo ? 'block' :
'none';
      document.getElementById('logoUrl').value = g.logoUrl || '';
      document.getElementById('logoSize').value = g.logoSize || 40;
      document.getElementById('logoPos').value = g.logoPos || 'left';
    }
  }

```

```

        document.getElementById('animEntrada').value = g.animEntrada ||
'slideLeft';
        document.getElementById('animSalida').value = g.animSalida || 'slideLeft';

        document.getElementById('usarCiclo').checked = !!g.usarCiclo;
        document.getElementById('timerSeconds').value = g.timerSeconds || 5;
        document.getElementById('tiempoOculto').value = g.tiempoOculto || 3;

        document.getElementById('lblX').textContent = Math.round(g.posX);
        document.getElementById('lblY').textContent = Math.round(g.posY);
    }

    // 3. Render Previo
    const stage = document.getElementById('stage');
    stage.innerHTML = '';
    Object.values(gafetesLocales).forEach(item => {
        const box = document.createElement('div');
        box.className = `prev-box logo-${item.logoPos} || 'left'` ${item.id ===
idSeleccionado ? 'selected' : ''}`;
        box.style.left = item.posX + '%';
        box.style.top = item.posY + '%';

        let imgHtml = '';
        if (item.usarLogo && item.logoUrl) {
            imgHtml = ``;
        }

        box.innerHTML = `
            ${imgHtml}
            <div>
                <div style="font-weight:bold; font-size:${(item.size1||28)/2.5}px;
color:${item.color1};">${item.linea1}</div>
                <div style="font-size:${(item.size2||16)/2.5}px;
color:${item.color2};">${item.linea2}</div>
            </div>
        `;

        box.onmousedown = (e) => {
            idSeleccionado = item.id;
            dragging = true;
            render();
        };

        stage.appendChild(box);
    });

    // 4. Botonera
    const bot = document.getElementById('botonera');
```

```

bot.innerHTML = '';
Object.values(gafetesLocales).forEach(item => {
  const div = document.createElement('div');
  div.className = 'bot-item';
  const estaActivo = item.visibilidad;

  div.innerHTML = `
    <strong>${item.nombreGafete}</strong> ${item.usarCiclo ? '🔄' : ''}
    <button class="btn-gray ${estaActivo ? 'active' : ''}"
onclick="socket.emit('toggle-visibility', '${item.id}')">
      ${estaActivo ? '🔴 EN PANTALLA (Clic para Ocultar)' : '🔵 MOSTRAR EN
OBS'}
    </button>
  `;
  bot.appendChild(div);
});
}

```

// Drag & Drop

```

const stage = document.getElementById('stage');
stage.onmousemove = (e) => {
  if (!dragging || !idSeleccionado) return;
  const rect = stage.getBoundingClientRect();
  let x = ((e.clientX - rect.left) / rect.width) * 100;
  let y = ((e.clientY - rect.top) / rect.height) * 100;

  x = Math.max(0, Math.min(100, x));
  y = Math.max(0, Math.min(100, y));

  gafetesLocales[idSeleccionado].posX = x;
  gafetesLocales[idSeleccionado].posY = y;

  document.getElementById('lblX').textContent = Math.round(x);
  document.getElementById('lblY').textContent = Math.round(y);

  actualizar();
};

```

```

window.onmouseup = () => { dragging = false; };

```

```

function seleccionarGafete(id) {
  idSeleccionado = id;
  render();
}

```

```

function actualizar() {
  if (!idSeleccionado) return;
  const data = {
    id: idSeleccionado,
    nombreGafete: document.getElementById('nombreGafete').value,
  };
}

```

```

    linea1: document.getElementById('linea1').value,
    size1: Number(document.getElementById('size1').value),
    color1: document.getElementById('color1').value,
    linea2: document.getElementById('linea2').value,
    size2: Number(document.getElementById('size2').value),
    color2: document.getElementById('color2').value,
    usarLogo: document.getElementById('usarLogo').checked,
    logoUrl: document.getElementById('logoUrl').value,
    logoSize: Number(document.getElementById('logoSize').value),
    logoPos: document.getElementById('logoPos').value,
    animEntrada: document.getElementById('animEntrada').value,
    animSalida: document.getElementById('animSalida').value,
    usarCiclo: document.getElementById('usarCiclo').checked,
    timerSeconds: Number(document.getElementById('timerSeconds').value),
    tiempoOculto: Number(document.getElementById('tiempoOculto').value),
    posX: gafetesLocales[idSeleccionado].posX,
    posY: gafetesLocales[idSeleccionado].posY
};

```

```

    socket.emit('guardar-gafete', data);
}

```

```

function eliminarActual() {
    if (idSeleccionado) {
        socket.emit('eliminar-gafete', idSeleccionado);
        idSeleccionado = null;
    }
}

```

```

</script>

```

```

</body>

```

```

</html>

```

```

----- overlay.html

```

```

<!DOCTYPE html>

```

```

<html lang="es">

```

```

<head>

```

```

    <meta charset="UTF-8">

```

```

    <script src="/socket.io/socket.io.js"></script>

```

```

    <style>

```

```

        body {

```

```

            background-color: rgba(0, 0, 0, 0);

```

```

            margin: 0;

```

```

            overflow: hidden;

```

```

            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

```

```

            width: 100vw;

```

```

            height: 100vh;

```

```

            position: relative;

```

```

        }

```

```

/* POSICIONAMIENTO DINÁMICO EXACTO EN OBS */

```

```

.gafete-box {
  position: absolute;
  padding: 14px 24px;
  background: rgba(0, 0, 0, 0.85);
  border-left: 5px solid #2563eb;
  border-radius: 8px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.6);
  display: flex;
  align-items: center;
  gap: 12px;
  transform: translate(-50%, -50%); /* Centra el punto de origen en posX, posY
*/
  pointer-events: none;
  opacity: 0;
}

/* POSICIONES DEL LOGO */
.logo-top { flex-direction: column; text-align: center; }
.logo-bottom { flex-direction: column-reverse; text-align: center; }
.logo-right { flex-direction: row-reverse; }
.logo-left { flex-direction: row; }

/* ESTADOS DE VISIBILIDAD */
.hidden { display: none !important; }

/* ANIMACIONES DE ENTRADA */
.in-slideLeft { animation: slideLeftIn 0.5s forwards ease-out; }
.in-slideRight { animation: slideRightIn 0.5s forwards ease-out; }
.in-slideUp { animation: slideUpIn 0.5s forwards ease-out; }
.in-bounce { animation: bounceIn 0.6s forwards cubic-bezier(0.68, -0.55, 0.265,
1.55); }
.in-zoom { animation: zoomIn 0.4s forwards ease-out; }
.in-fadeIn { animation: fadeIn 0.5s forwards; }

/* ANIMACIONES DE SALIDA */
.out-slideLeft { animation: slideLeftOut 0.5s forwards ease-in; }
.out-slideRight { animation: slideRightOut 0.5s forwards ease-in; }
.out-slideUp { animation: slideUpOut 0.5s forwards ease-in; }
.out-zoom { animation: zoomOut 0.4s forwards ease-in; }
.out-fadeOut { animation: fadeOut 0.5s forwards; }

/* KEYFRAMES */
@keyframes fadeIn { to { opacity: 1; } }
@keyframes fadeOut { from { opacity: 1; } to { opacity: 0; } }

@keyframes slideLeftIn { from { transform: translate(calc(-50% - 150px), -50%);
opacity: 0; } to { transform: translate(-50%, -50%); opacity: 1; } }
@keyframes slideLeftOut { from { transform: translate(-50%, -50%); opacity: 1;
} to { transform: translate(calc(-50% - 150px), -50%); opacity: 0; } }

```

```

    @keyframes slideRightIn { from { transform: translate(calc(-50% + 150px),
-50%); opacity: 0; } to { transform: translate(-50%, -50%); opacity: 1; } }
    @keyframes slideRightOut { from { transform: translate(-50%, -50%); opacity: 1;
} to { transform: translate(calc(-50% + 150px), -50%); opacity: 0; } }

    @keyframes slideUpIn { from { transform: translate(-50%, calc(-50% + 100px));
opacity: 0; } to { transform: translate(-50%, -50%); opacity: 1; } }
    @keyframes slideUpOut { from { transform: translate(-50%, -50%); opacity: 1; }
to { transform: translate(-50%, calc(-50% + 100px)); opacity: 0; } }

    @keyframes bounceIn {
        0% { transform: translate(-50%, -50%) scale(0.3); opacity: 0; }
        50% { transform: translate(-50%, -50%) scale(1.05); opacity: 1; }
        100% { transform: translate(-50%, -50%) scale(1); opacity: 1; }
    }

    @keyframes zoomIn { from { transform: translate(-50%, -50%) scale(0); opacity:
0; } to { transform: translate(-50%, -50%) scale(1); opacity: 1; } }
    @keyframes zoomOut { from { transform: translate(-50%, -50%) scale(1); opacity:
1; } to { transform: translate(-50%, -50%) scale(0); opacity: 0; } }
</style>
</head>
<body>

    <div id="container"></div>

    <script>
        const socket = io();

        socket.on('sync-gafetes', (gafetes) => {
            const container = document.getElementById('container');

            Object.values(gafetes).forEach(g => {
                let el = document.getElementById(g.id);

                if (!el) {
                    el = document.createElement('div');
                    el.id = g.id;
                    el.className = 'gafete-box hidden';
                    el.innerHTML = `
                        <img class="g-img" style="display:none; object-fit:contain;">
                        <div>
                            <div class="g-l1" style="font-weight:800; line-height:1.2;"></div>
                            <div class="g-l2" style="margin-top:4px; line-height:1.2;"></div>
                        </div>
                    `;
                    container.appendChild(el);
                }

                // Aplicar posición porcentual real

```

```

el.style.left = g.posX + '%';
el.style.top = g.posY + '%';

// Aplicar clase de alineación de logo
el.className = el.className.replace(/logo-\w+/g, '');
el.classList.add(`logo-${g.logoPos} || 'left'`);

// Línea 1 y 2
const l1 = el.querySelector('.g-l1');
l1.textContent = g.linea1;
l1.style.fontSize = g.size1 + 'px';
l1.style.color = g.color1;

const l2 = el.querySelector('.g-l2');
l2.textContent = g.linea2;
l2.style.fontSize = g.size2 + 'px';
l2.style.color = g.color2;

// MANEJO DE LOGO (SOLO SI TIENE EL CHECK ACTIVADO)
const img = el.querySelector('.g-img');
if (g.usarLogo && g.logoUrl) {
  if (img.src !== g.logoUrl) img.src = g.logoUrl;
  img.style.display = 'block';
  img.style.width = (g.logoSize || 40) + 'px';
  img.style.height = (g.logoSize || 40) + 'px';
} else {
  img.style.display = 'none';
  img.removeAttribute('src'); // Evita errores de carga si no hay logo
}

// LÓGICA DE ANIMACIÓN DE ENTRADA Y SALIDA
if (g.visibilidad) {
  if (!el.classList.contains('is-visible')) {
    el.classList.remove('hidden', `out-${g.animSalida}`);
    el.classList.add('is-visible', `in-${g.animEntrada}`);
  }
} else {
  if (el.classList.contains('is-visible')) {
    el.classList.remove('is-visible', `in-${g.animEntrada}`);
    el.classList.add(`out-${g.animSalida}`);
    setTimeout(() => {
      if (!el.classList.contains('is-visible')) {
        el.classList.add('hidden');
      }
    }, 500);
  }
}
});
});

```

```
    socket.on('reproducir-sonido', (sonido) => {  
      new Audio(`sonidos/${sonido}.mp3`).play().catch(e => console.error(e));  
    });  
</script>  
</body>  
</html>
```